Diffusion speculators

Arseny Rybakov, 09.09.2026

DISCLAIMER!

MOST OF BENCHMARKS ARE DONE ONE IN VERY LOW-CONCURRENCY REGIME OF THE INFERENCE ENGINE!!!

Plan

Classic: Dflash and DFlare

The main idea

Additions

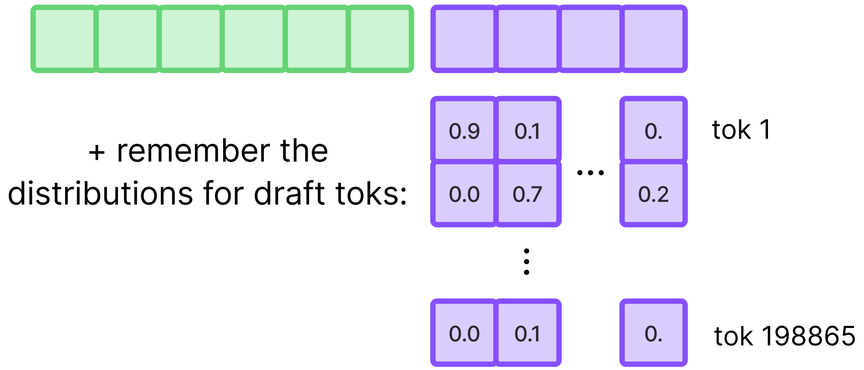
Controlling draft length, drafted distribution post-processing

Monsters

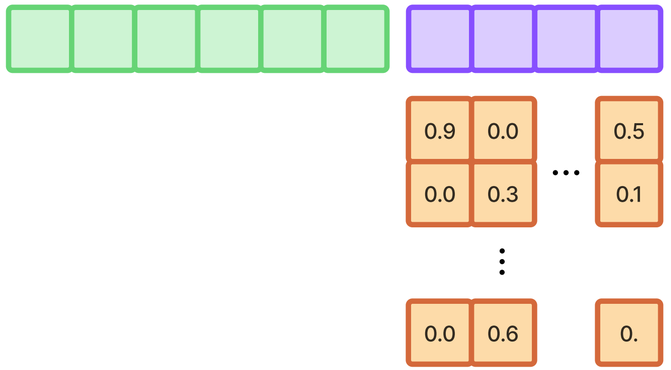
Verifier and drafter are a single model somehow

Speculate with diffusion

Take the initial prefix and draft k tokens conditioned on it



Get the target model distribution for every position of the draft



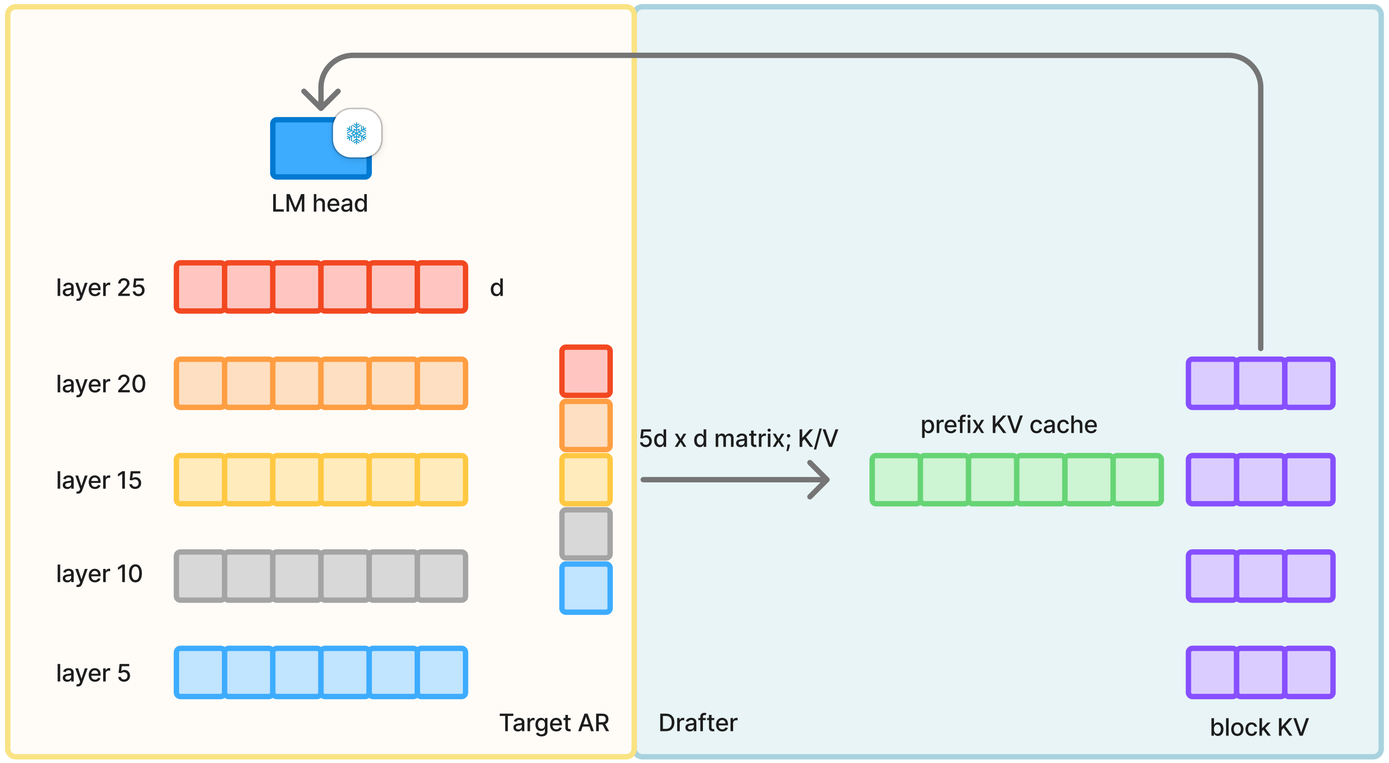
Accept all tokens till the first rejected token

Resample rejected token and drop the rest

$$p_{acc} = \min\left(1, \frac{p(x)}{q(x)}\right)$$

$$tok_{resampled} \sim \frac{\max(0, p(x) - q(x))}{\sum_i \max(0, p(x_i) - q(x_i))}$$

DFlash



- 1-step masked diffusion trained from scratch
- for conditioning on prefix, we take prefix hiddens from several layers, project them to the drafter hidden dimension, then project it via K and matrices and treat this as a KV cache for every drafter layer
- token embeddings are not learned and originate from the target LM
- loss: CE with the target AR model outputs; tokens are exponentially weighted by their positions:

$$w_k = \exp\left(-\frac{k-1}{\gamma}\right)$$

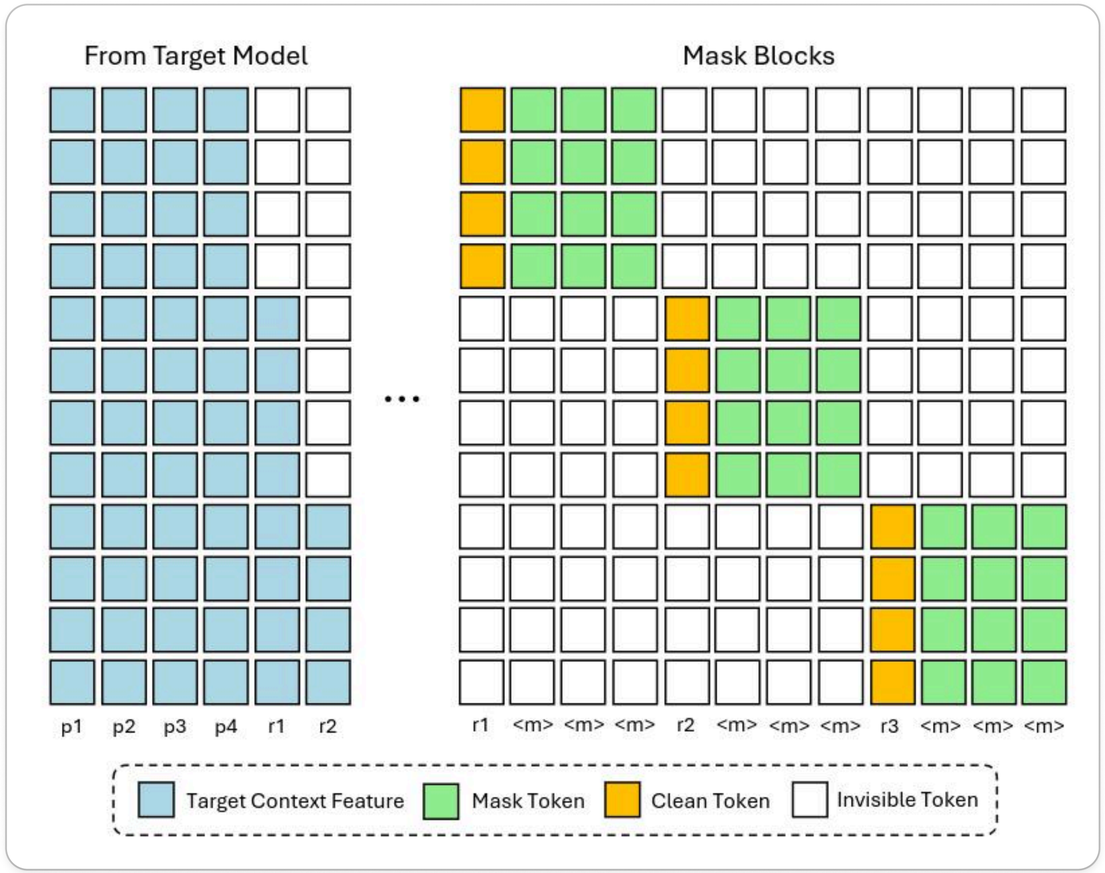
DFlash

Z Lab

Training:

- prefill a query-response pair from some instruction dataset (think codealpaca)
- randomly sample positions from which drafter blocks will start
- add [MASK] tokens and assign them correct positions

- Let FlexAttention do the rest



DFlash

Z Lab

Benchmarks to show off:

Table 1. Decoding speedup over baseline and average acceptance length (τ) on Qwen3 models with thinking mode disabled and a maximum of 2048 generated tokens. Parenthesized values indicate the draft tree size for EAGLE-3 and the diffusion block size for DFlash.

Model	Method	MATH						CODE						CHAT			
		GSM8K		MATH-500		AIME25		HumanEval		MBPP		LCB		MT-Bench		*Avg.*	
Temperature = 0		Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Q3-4B	EAGLE-3 (16)	1.99×	3.30	1.83×	3.08	1.79×	3.05	1.84×	3.05	1.78×	2.95	1.73×	2.91	1.74×	3.02	1.81×	3.05
	EAGLE-3 (60)	2.27×	3.77	2.10×	3.52	2.13×	3.51	2.12×	3.47	2.02×	3.38	1.90×	3.22	2.04×	3.49	2.08×	3.48
	DFlash (16)	**5.15×**	**6.53**	**6.09×**	**7.84**	**5.68×**	**7.27**	**5.21×**	**6.64**	**4.78×**	**6.09**	**5.41×**	**7.09**	**2.85×**	**4.35**	**4.91×**	**6.54**
Q3-8B	EAGLE-3 (16)	1.94×	3.23	1.81×	3.02	1.79×	3.00	1.89×	3.17	1.69×	2.82	1.57×	2.65	1.63×	2.83	1.76×	2.96
	EAGLE-3 (60)	2.23×	3.71	2.05×	3.49	2.05×	3.44	2.17×	3.65	1.93×	3.25	1.81×	3.03	1.90×	3.26	2.02×	3.40
	DFlash (16)	**5.15×**	**6.54**	**6.08×**	**7.87**	**5.62×**	**7.08**	**5.14×**	**6.50**	**4.65×**	**5.95**	**5.51×**	**7.27**	**2.75×**	**4.24**	**4.86×**	**6.49**
Temperature = 1		Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Q3-4B	EAGLE-3 (16)	1.89×	3.22	1.75×	2.99	1.64×	2.79	1.74×	3.01	1.69×	2.89	1.63×	2.77	1.70×	2.95	1.72×	2.95
	EAGLE-3 (60)	2.12×	3.68	1.97×	3.44	1.83×	3.20	1.94×	3.39	1.92×	3.33	1.82×	3.14	1.91×	3.36	1.93×	3.36
	DFlash (16)	**4.71×**	**6.00**	**5.09×**	**6.67**	**3.73×**	**4.92**	**4.74×**	**6.04**	**4.42×**	**5.66**	**4.90×**	**6.50**	**2.67×**	**4.07**	**4.24×**	**5.69**
Q3-8B	EAGLE-3 (16)	1.87×	3.12	1.73×	2.91	1.63×	2.74	1.75×	3.05	1.64×	2.74	1.56×	2.57	1.58×	2.70	1.68×	2.83
	EAGLE-3 (60)	2.07×	3.59	1.94×	3.38	1.84×	3.18	2.05×	3.54	1.85×	3.16	1.72×	2.92	1.70×	3.05	1.88×	3.26
	DFlash (16)	**4.67×**	**5.98**	**4.84×**	**6.40**	**3.57×**	**4.73**	**4.32×**	**5.52**	**4.04×**	**5.21**	**4.93×**	**6.69**	**2.47×**	**3.80**	**4.03×**	**5.48**

DFlash

Z Lab

Benchmarks to actually think about:

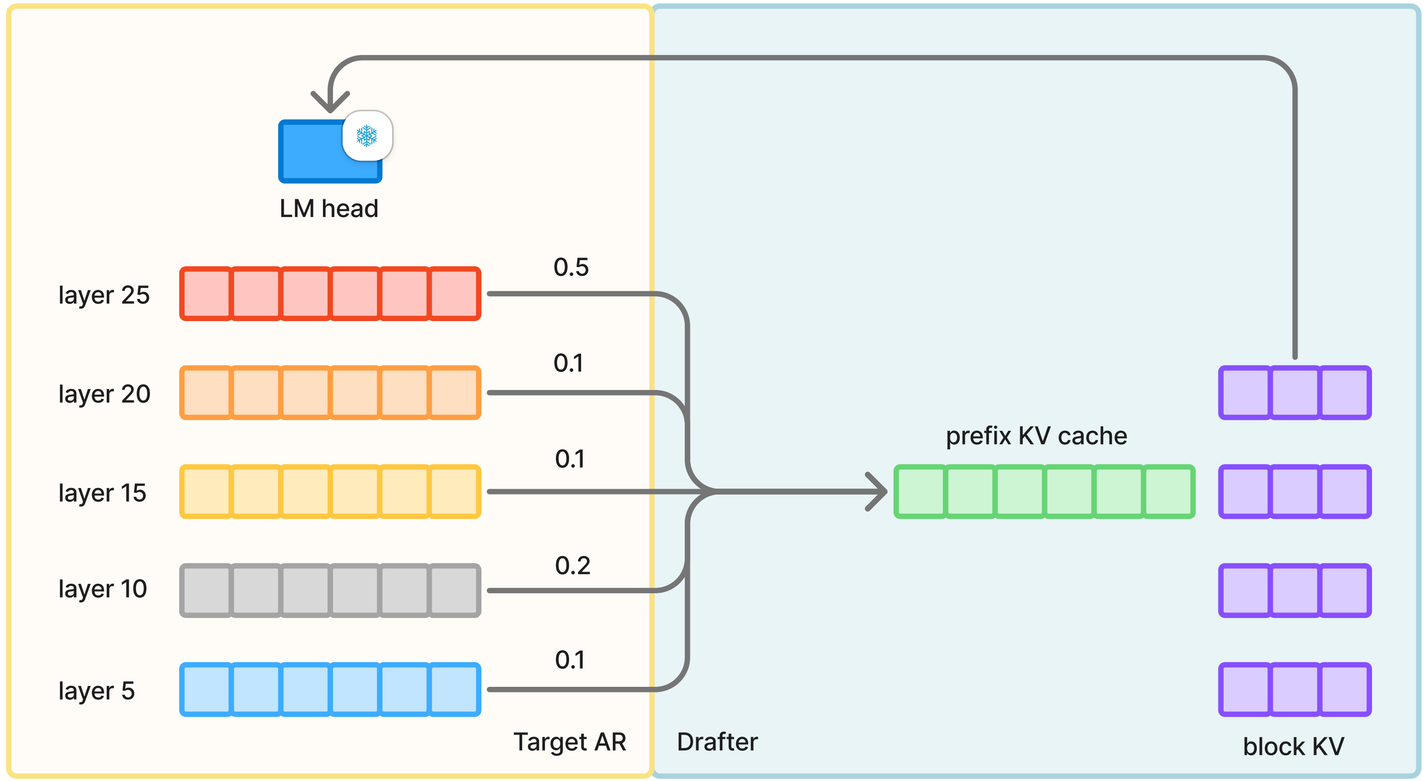
Table 3. Throughput (tok/s), speedup over baseline, and average acceptance length τ on SGLang (FA4 backend).

Task	Method	Concurrency 1	4	8	16	32	*Avg.* τ
Qwen3-4B							
Math500	Baseline	316	1145	2201	4100	7136	–
	DFlash	1531 4.8×	4943 4.3×	9066 4.1×	14477 3.5×	20417 2.9×	8.01
Human-Eval	Baseline	312	1162	2217	4184	7143	–
	DFlash	1247 4.0×	4147 3.6×	6997 3.2×	11234 2.7×	15703 2.2×	6.63
Qwen3-8B							
Math500	Baseline	230	861	1666	3133	5694	–
	DFlash	1175 5.1×	3884 4.5×	7485 4.5×	12268 3.9×	16076 2.8×	8.01
Human-Eval	Baseline	229	868	1649	3253	5462	–
	DFlash	955 4.2×	3092 3.6×	6010 3.6×	9919 3.0×	13116 2.4×	6.50

Table 5. Speedup over baseline and average acceptance length τ for LLaMA-3.1-8B-Instruct on SGLang (Flashinfer backend, single B200 GPU). Baseline reports absolute throughput (TPS; tokens/s). EAGLE-3 uses 7 draft steps with top-k=10 and either 10 or 60 draft tokens. DFlash uses block size 10.

Method	Concurrency 1	4	8	16	32	*Avg.* τ
GSM8K						
Baseline (TPS)	249	923	1739	3245	5349	–
EAGLE-3 (10)	1.6×	1.5×	1.4×	1.2×	1.0×	3.49
EAGLE-3 (60)	1.9×	1.6×	1.3×	0.9×	0.6×	4.55
DFlash (10)	**2.4×**	**2.2×**	**2.1×**	**1.8×**	**1.6×**	**4.32**
HumanEval						
Baseline (TPS)	245	922	1778	3336	5854	–
EAGLE-3 (10)	2.0×	1.9×	1.8×	1.5×	1.2×	3.62
EAGLE-3 (60)	2.0×	1.7×	1.3×	0.9×	0.6×	4.65
DFlash (10)	**2.8×**	**2.6×**	**2.5×**	**2.1×**	**1.8×**	**4.91**
Alpaca						
Baseline (TPS)	245	906	1745	3237	5434	–
EAGLE-3 (10)	1.5×	1.4×	1.4×	1.1×	0.9×	3.11
EAGLE-3 (60)	1.8×	1.5×	1.2×	0.8×	0.5×	4.07
DFlash (10)	**2.2×**	**2.0×**	**1.8×**	**1.5×**	**1.4×**	**3.73**

DFlare



- DFlash, but let's blend hiddens that will represent the prefix, weights for blending are learnable
- Different K and V matrices for projecting draft tokens and context features:

$$\mathbf{K} = \left[\mathbf{F}^{(i)}\mathbf{W}_K^t \;;\; \mathbf{x}_t\,\mathbf{W}_K^d \;;\; \hat{\mathbf{X}}\,\mathbf{W}_K^d\right]$$
$$\mathbf{V} = \left[\mathbf{F}^{(i)}\mathbf{W}_V^t \;;\; \mathbf{x}_t\,\mathbf{W}_V^d \;;\; \hat{\mathbf{X}}\,\mathbf{W}_V^d\right]$$

- gamma from the loss now progressively increases the weights for draft suffix:

$$\gamma(s) = \gamma_0 + \frac{s}{S}\,(\gamma_{\max} - \gamma_0),$$
$$w_k(s) = \exp\left(-\frac{k-1}{\gamma(s)}\right),$$

DFlare

Core claim is that such design choice enables stacking more layers

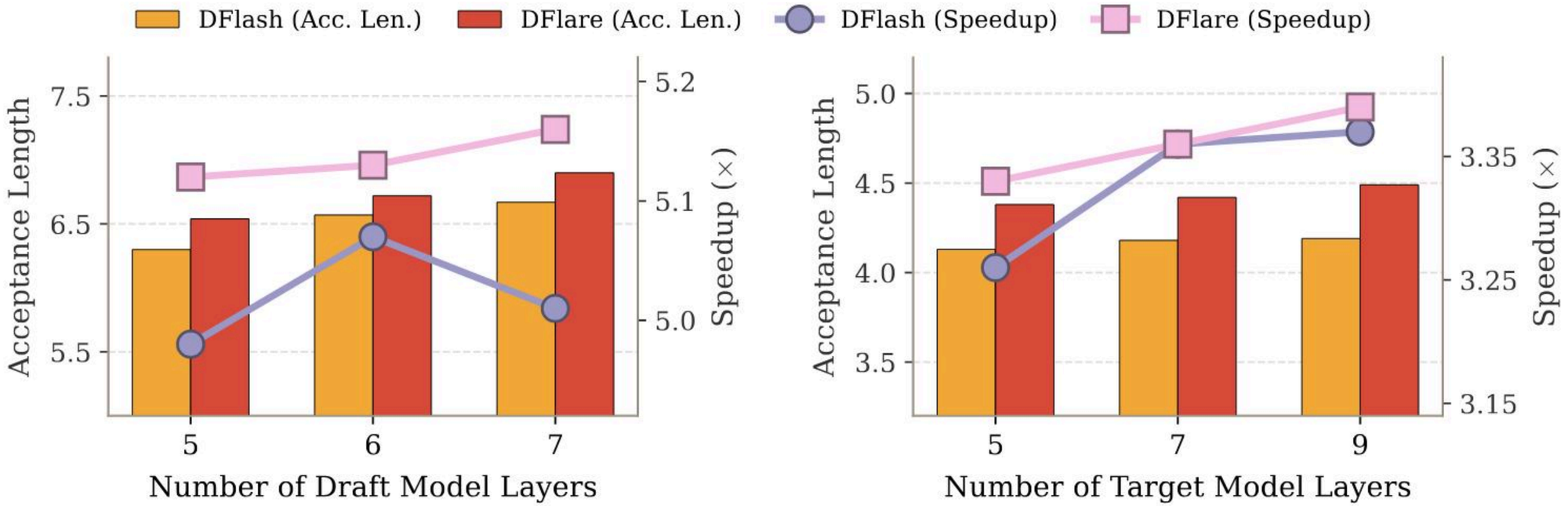


Figure 3: The impact of draft model layers (Left) and target model layers (Right) on the performance of DFlash and DFLARE . The target model is Qwen3-4B.

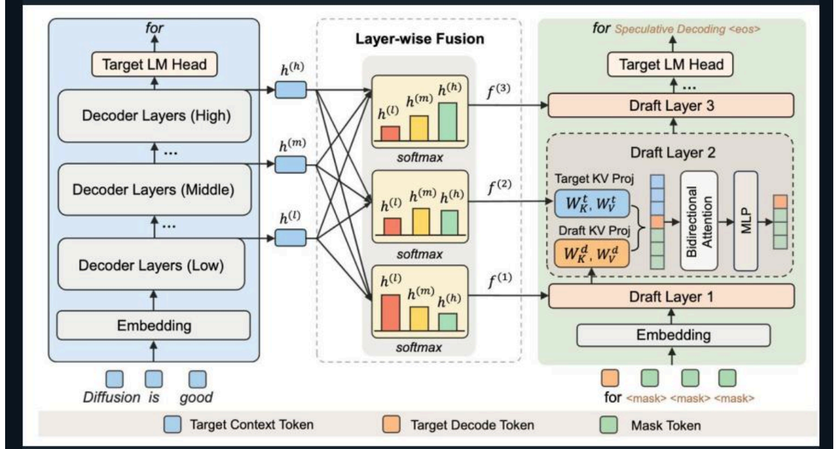
DFlare

Setting	GSM8k		Humaneval		MtBench		Avg	
	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
DFLARE	**3.80**	**4.94**	3.55	4.62	2.76	4.00	**3.37**	**4.52**
− Softmax	3.70	4.85	3.45	4.53	2.77	4.02	3.31	4.47
− KVProj	3.77	4.90	3.50	4.54	2.78	3.99	3.35	4.48
− Loss	3.72	4.84	**3.56**	**4.63**	2.73	3.93	3.34	4.47
+ FC	3.74	4.87	3.53	4.59	**2.78**	**4.06**	3.35	4.51

Table 2: Ablation on model structure design with Qwen3-4B as target model. Best results in each column are in **bold**.

КПД
3,451 subscribers

КПД



DFlare: Scaling Up Draft Capacity for Block Diffusion Speculative Decoding
[Статья]

По существу работа предлагает инкрементальные улучшения поверх DFlash (наткнулся, когда искал код FLARE), поэтому кратко:
👉 Вместо того, чтобы брать равномерно признаки с какого-то по какой-то слой, делают взвешенное смешивание с обучаемой матрицей.
👉 Разделяют KV проекции для target контекста и драфтовых токенов.
👉 В экспоненциальном затухании весов токенов при удалении вглубь драфта постепенно повышают знаменатель (снижают скорость затухания).
👉 Stack more layers and train on more (3x) data 😝.

В итоге оно дает 5-10% ускорения против DFlash.

👍 9 🤔 4 1768 edited 12:39 PM

3 comments

How to further improve the speed?

Target AR model +DFlash

Adaptive draft length

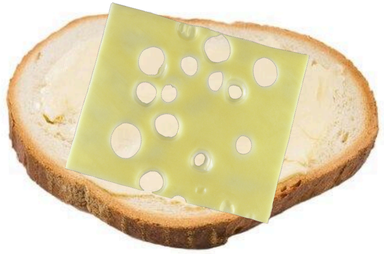
DSpark
BlockPilot

Draft trees

DDTree [suffer!]

+Ngram models

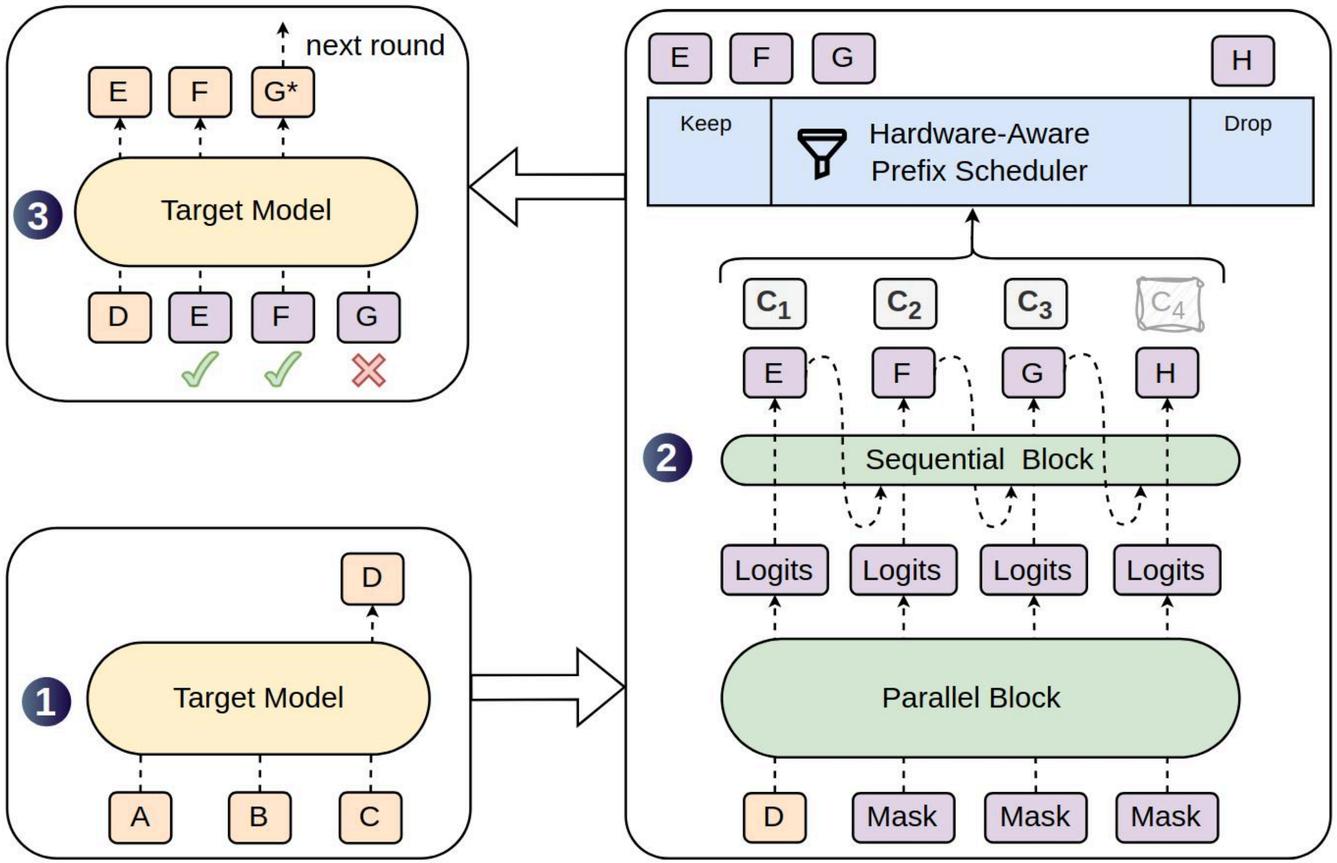
DART



Recycle rejected tokens

ReTrace

DSpark



- Take DFlash
- learn an additive correction to its logits using some sequential model:

$$P(X \mid x_0) = \prod_{k=1}^{\gamma} p_k(x_k \mid x_0, x_{<k}),$$

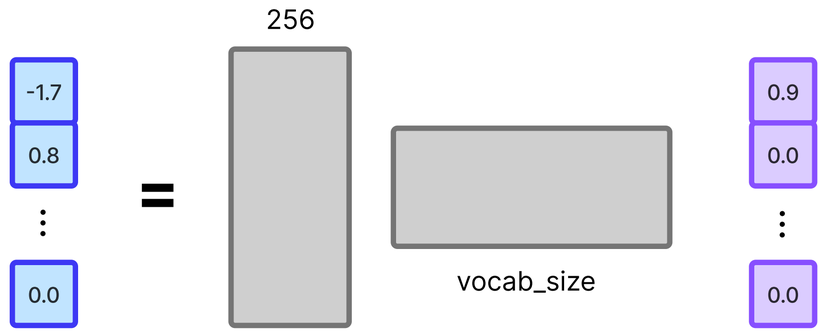
$$p_k(v \mid x_0, x_{<k}) = \frac{\exp(U_k(v) + B_k(x_0, x_{<k}, v))}{\sum_{u \in \mathcal{V}} \exp(U_k(u) + B_k(x_0, x_{<k}, u))}$$

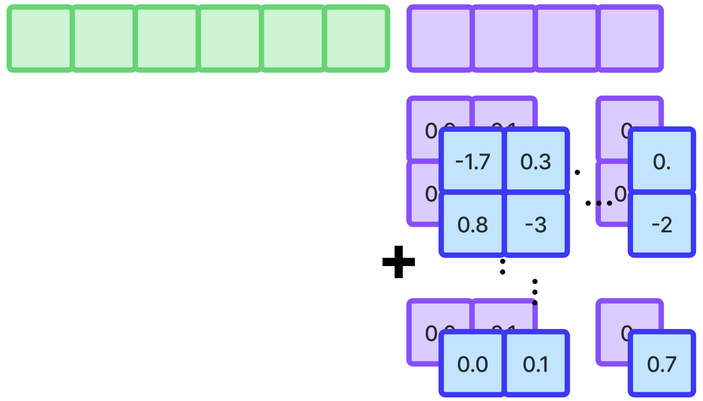
- confidence-based adaptive kept draft length

DSpark

What is this autoregressive correction?

Markov head $B(x_{k-1}, \cdot) = W_1[x_{k-1}]\, W_2 \in \mathbb{R}^V$





RNN head

$$s_k = \sigma(W_g\, z_k) \odot s_{k-1} + \big(1 - \sigma(W_g\, z_k)\big) \odot \tanh(W_c\, z_k),$$
$$B_k(x_{<k}, \cdot) = W_2^{\top} \tanh(W_o\, z_k),$$

DSpark

Confidence heads?

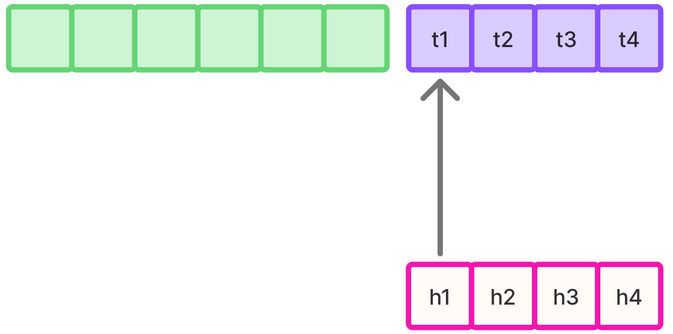
- Take the drafter hidden for the current token and the projected logits from the previous token
- project and activate:

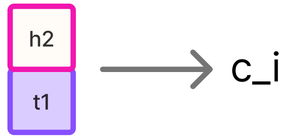
$$c_k = \sigma\left(w^\top [h_k;\, W_1 [x_{k-1}]]\right)$$

- supervise the confidence head via total variation distance:

$$c_k^* = 1 - \frac{1}{2}\|p_k^d - p_k^t\|_1$$

- in the end, tune the temperature on a held-out set of samples so confidence is close to real acceptance probability

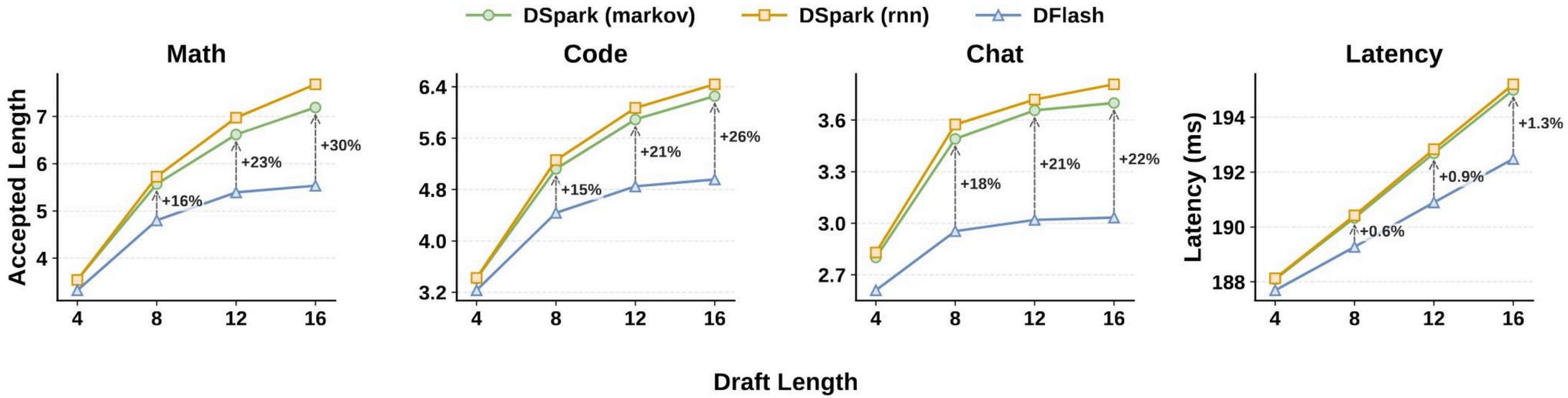




$$c_k = \sigma\left(w^\top [\text{h2}\; W_1 [x\; \text{t1}]]\right)$$

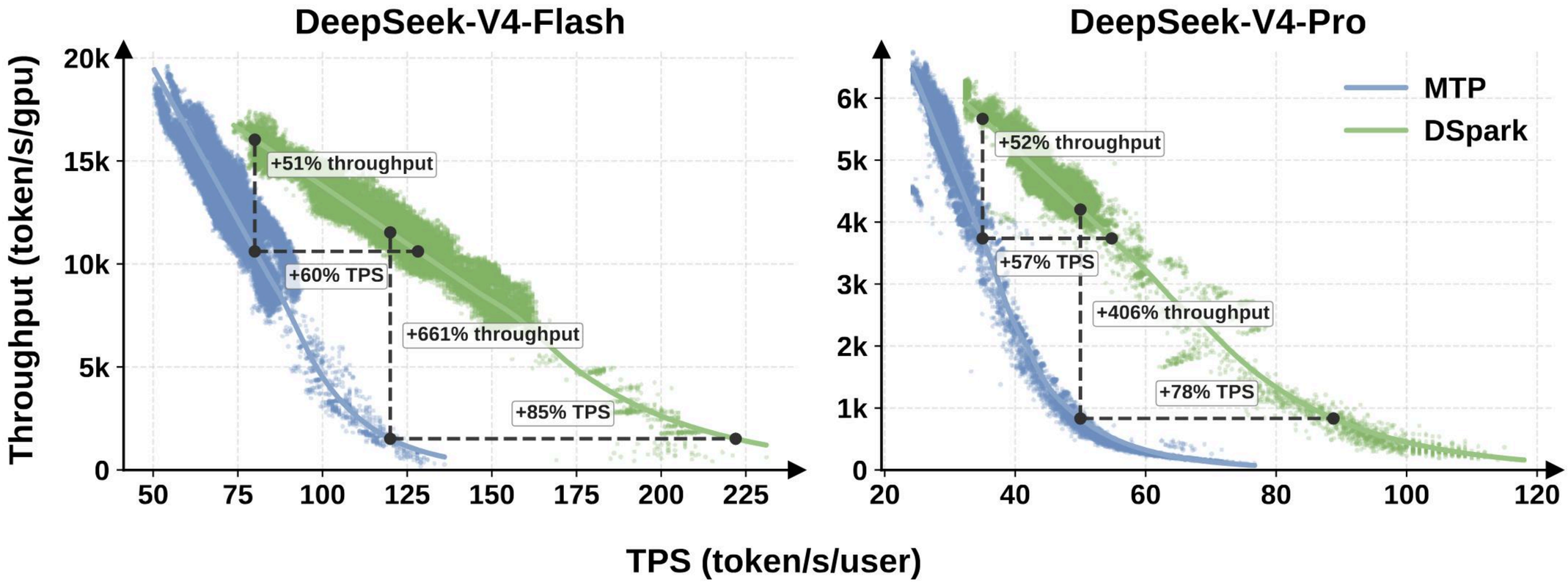
DSpark

Markov head works almost as good as RNN, at least for small Qwen3



DSpark

Confidence is used to decide how far to verify under the current compute budget



???

DSpark’s confidence allows us to choose how much to verify, but what if we want to choose how much to draft?

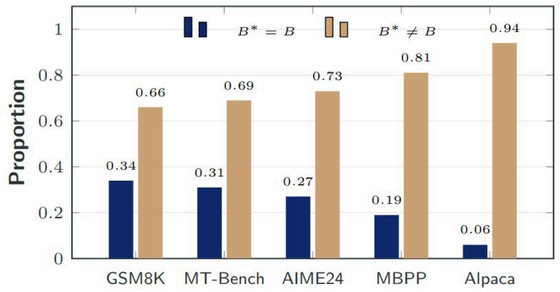
Then we can just train a model that predicts accepted draft length directly and use it, easy.

Sadly, there is no paper which does directly this. Instead, we have a monster built by chinese maps company.

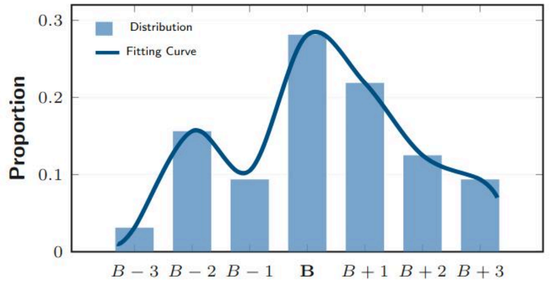
BlockPilot

DSpark's confidence allows us to choose how much to verify, but what if we want to choose how much to draft?

Consider we want to select the block size that → max accepted prefix; from this point of view, original block size is rarely optimal:



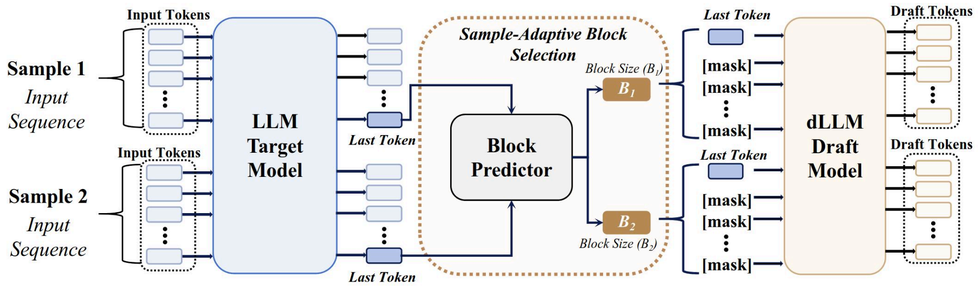
(a) Proportion of samples with B^* matching or mismatching B.



(b) Unimodal distribution of B^* peaking at the trained size B.

Then all we need to do is to try different block sizes for every context from the training data, select the best block size and let the predictor learn it.

Since we have not too much possible block sizes, why not to use a classifier instead of regressor.



ReTrace

ByteDance

After verification, we throw away the rejected tokens alongside with the features from target model for these tokens.

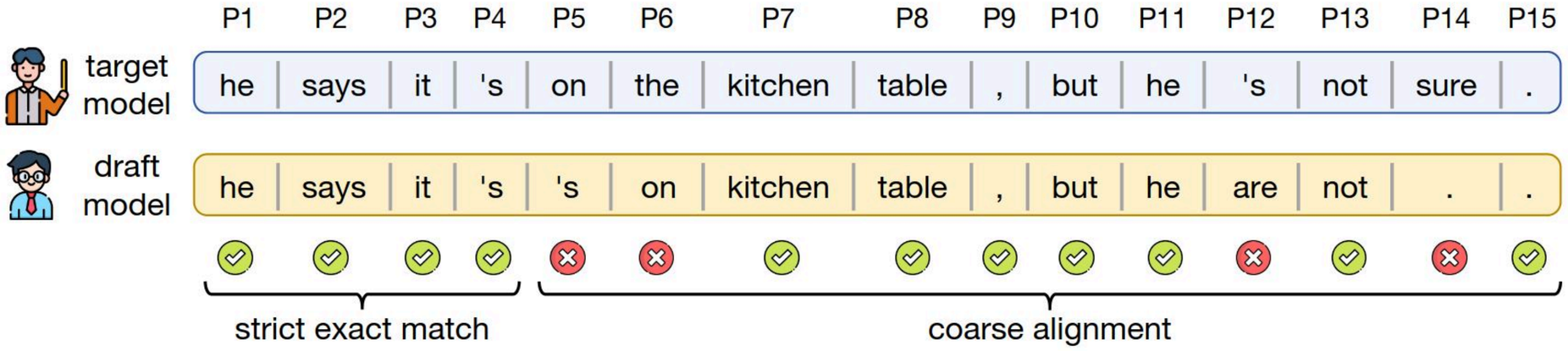
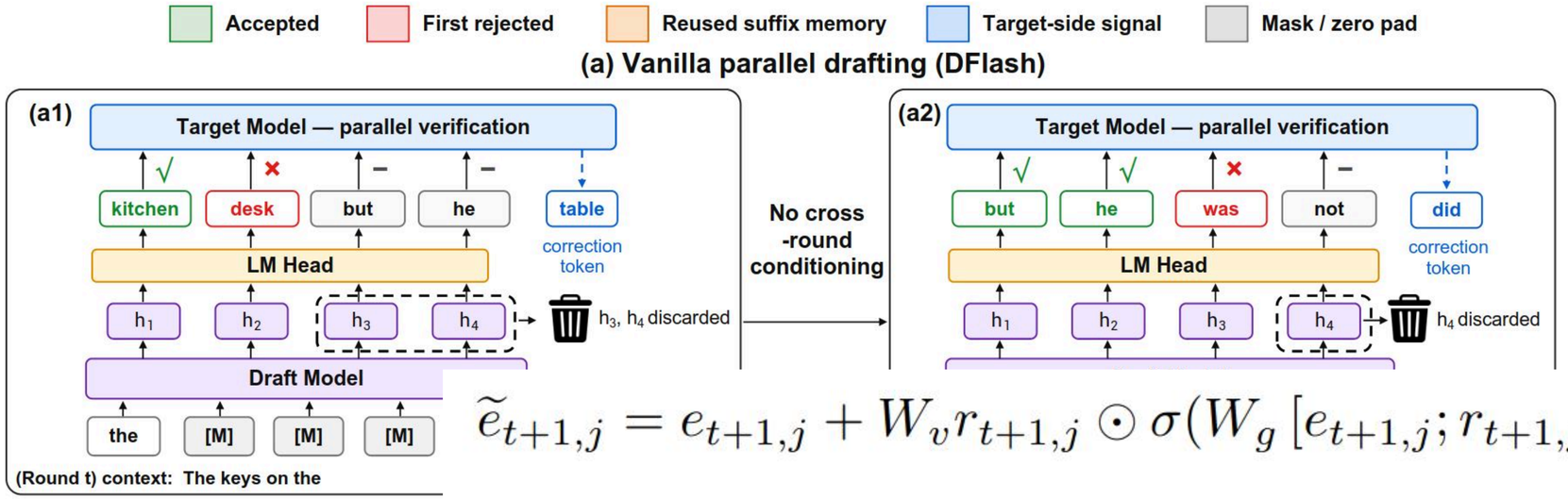


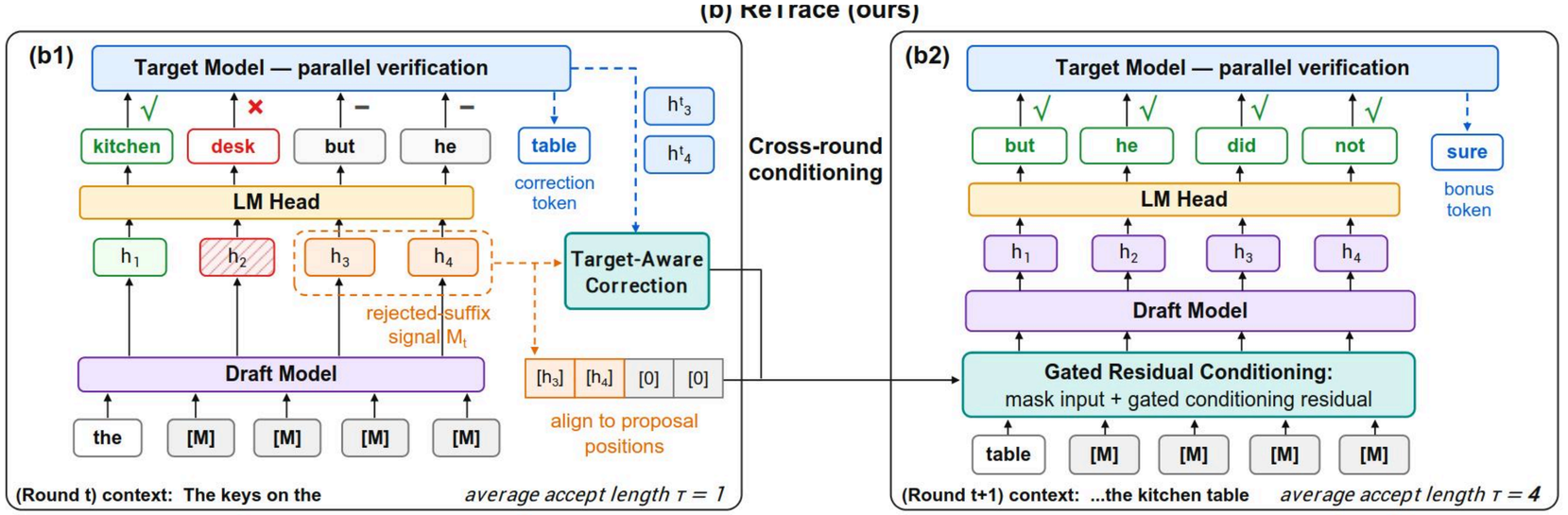
Figure 1 An MT-Bench example with Qwen3-4B as the target model. Although verification stops at the first mismatch, later draft tokens realign with the target, suggesting that the rejected suffix retains useful predictive information.

ReTrace

ByteDance

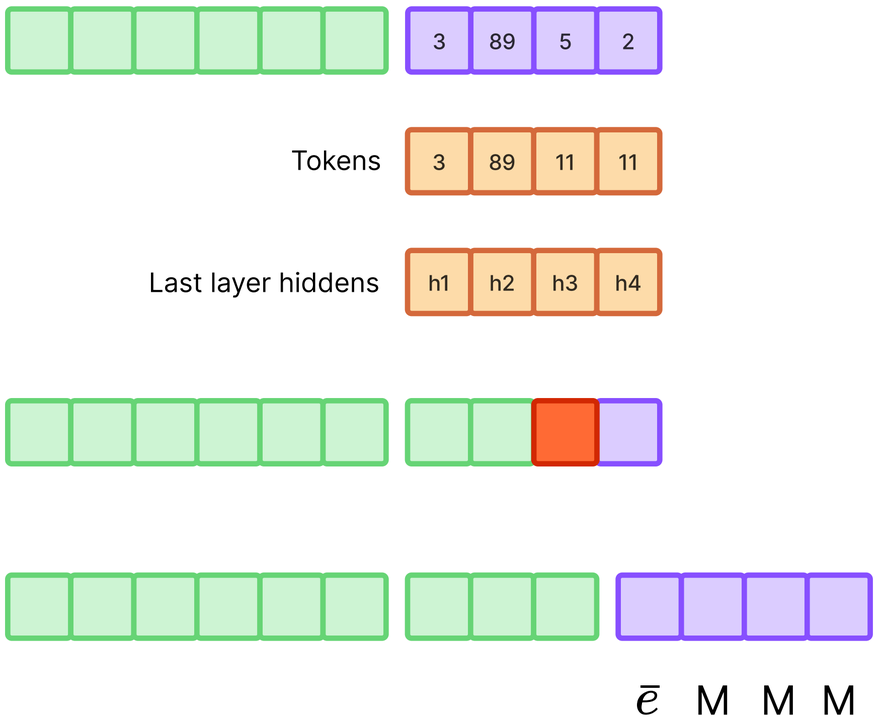


$$\widetilde{e}_{t+1,j} = e_{t+1,j} + W_v r_{t+1,j} \odot \sigma\left(W_g \left[e_{t+1,j}; r_{t+1,j}\right]\right), \tag{9}$$



ReTrace

Update the initial point for the rejected token, but how?



$$c_{t+1,j} = W_c \left[r \boxed{\text{h4}}_j ; \boxed{\text{h4}} \right],$$

$$r_{t+1,j} = r \boxed{\text{h4}}_j + \beta\, c_{t+1,j}.$$

$$\widetilde{e}_{t+1,j} = \boxed{\text{M}} + W_v \boxed{\text{M}} \odot \sigma(W_g \left[\boxed{\text{M}} ; r_{t+1,j} \right])$$

ReTrace ByteDance

Model	Method	Math						Code				Chat				Overall	
		GSM8K		MATH-500		AIME25		HumanEval		LCB		MT-Bench		Alpaca		*Avg.*	
Temperature = 0		Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Qwen3-4B	EAGLE-3 (16)	1.67×	3.31	1.67×	3.14	1.76×	3.25	1.58×	3.07	1.74×	3.03	1.42×	2.91	1.43×	2.88	1.61×	3.08
	EAGLE-3 (60)	1.67×	3.70	1.67×	3.49	1.71×	3.61	1.61×	3.37	1.81×	3.35	1.43×	3.21	1.42×	3.17	1.62×	3.41
	DFlash (16)	3.37×	6.36	5.18×	8.25	5.32×	8.66	4.15×	6.69	4.68×	7.35	1.85×	4.07	1.66×	3.48	3.74×	6.41
	ReTrace (16)	**3.73×**	**7.36**	**5.47×**	**8.88**	**5.44×**	**8.94**	**4.27×**	**6.90**	**4.85×**	**7.80**	**1.99×**	**4.40**	**1.73×**	**3.69**	**3.93×**	**6.85**
Qwen3-8B	EAGLE-3 (16)	1.85×	3.24	2.04×	3.15	2.23×	3.65	1.93×	3.21	1.86×	2.86	1.67×	2.73	1.50×	2.60	1.87×	3.06
	EAGLE-3 (60)	1.92×	3.61	2.18×	3.52	2.06×	3.65	1.96×	3.55	1.86×	3.14	1.71×	3.01	1.53×	2.80	1.89×	3.33
	DFlash (16)	3.91×	6.40	5.89×	8.21	5.96×	8.40	4.03×	6.49	4.26×	7.17	2.39×	4.18	1.89×	3.59	4.05×	6.35
	ReTrace (16)	**4.34×**	**7.39**	**6.10×**	**8.95**	**6.10×**	**8.73**	**4.21×**	**6.74**	**4.38×**	**7.50**	**2.47×**	**4.41**	**1.97×**	**3.81**	**4.22×**	**6.79**
Temperature = 1		Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Qwen3-4B	EAGLE-3 (16)	1.51×	3.26	1.38×	3.03	1.35×	2.86	1.39×	3.02	1.34×	2.99	1.30×	2.89	1.28×	2.82	1.36×	2.98
	EAGLE-3 (60)	1.49×	3.63	1.36×	3.38	1.30×	3.14	1.37×	3.30	1.37×	3.28	1.24×	3.14	1.25×	3.11	1.34×	3.28
	DFlash (16)	2.84×	6.00	3.23×	6.99	2.46×	5.18	2.96×	6.04	2.92×	7.04	1.59×	3.96	**1.45×**	3.43	2.49×	5.52
	ReTrace (16)	**3.15×**	**6.82**	**3.39×**	**7.50**	**2.60×**	**5.41**	**2.99×**	**6.30**	**3.56×**	**7.60**	**1.62×**	**4.16**	1.44×	**3.56**	**2.68×**	**5.91**
Qwen3-8B	EAGLE-3 (16)	1.69×	3.14	1.60×	2.94	1.52×	2.76	1.68×	3.10	1.47×	2.82	1.42×	2.67	1.36×	2.55	1.53×	2.85
	EAGLE-3 (60)	1.73×	3.51	1.58×	3.28	1.51×	3.06	1.69×	3.41	1.29×	3.12	1.39×	2.89	1.35×	2.74	1.51×	3.14
	DFlash (16)	3.15×	5.82	3.53×	6.81	2.99×	5.34	3.24×	5.63	3.52×	7.12	1.76×	3.73	1.63×	3.44	2.83×	5.41
	ReTrace (16)	**3.48×**	**6.63**	**3.95×**	**7.36**	**3.04×**	**5.58**	**3.32×**	**5.79**	**3.59×**	**7.40**	**1.82×**	**4.08**	**1.68×**	**3.69**	**2.98×**	**5.79**

DFlash2

inco.ai
inference, reimagined for the agentic era

Let’s solve inconsistency problem while still working in parallel!

- after we ran the drafter, let us pick top-16 candidates
- calculate S for every pair of sequential candidates:

$$S_t(a,b) = U_t(b) + \langle A(a) \odot H(h_t), B(b) \rangle.$$

- select the pair with the best score
- during the training, train this selector via CE: teacher-force the previous token and train selector to predict correct next token

$$\mathrm{Conv}_k(x)_t = k_{t,0} \odot x_t + k_{t,1} \odot x_{t-1}.$$

Each coefficient combines a learned base kernel with a small correction computed from the current hidden state; every 16 channels share one correction. The first position reads the last verified token's representation, and every later position reads its predecessor's. Information crosses the block while all positions still compute in parallel.

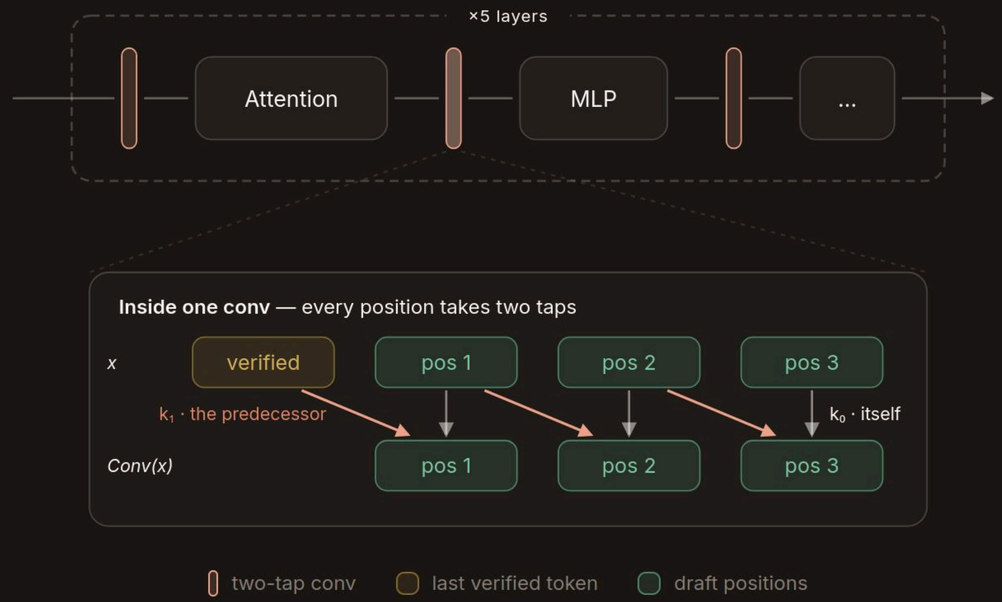


Figure 4. The two-tap dynamic convolution. One sits before and after each attention and MLP sublayer of every drafter layer. Inside it, each position mixes its own representation with its predecessor's, and the first position reads the last verified token.

DFlash2

inco.ai
inference, reimagined for the agentic era

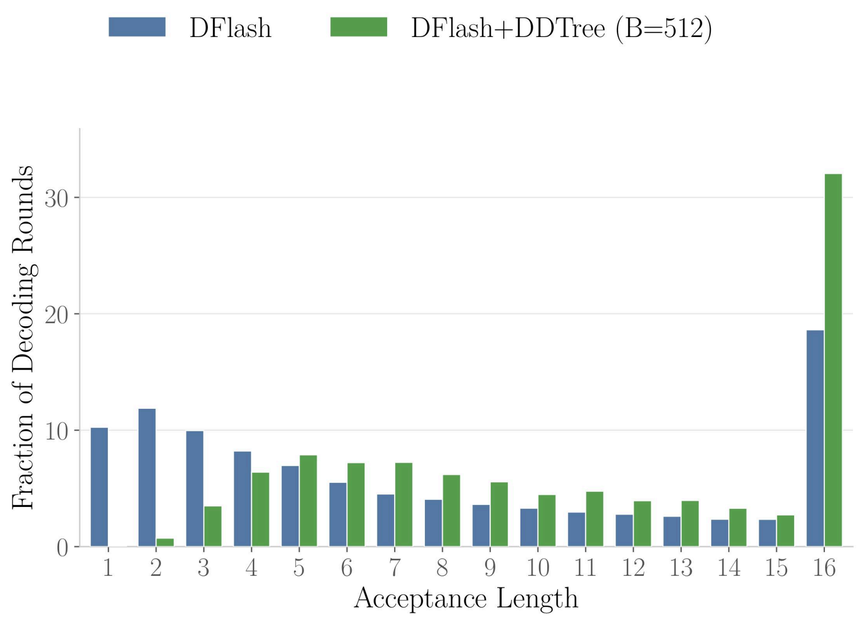
Qwen3.8-27B

DATASET	MTP	DSPARK	DFLASH 2
GSM8K	5.02	4.36	**5.46**
MATH-500	4.72	3.92	**5.28**
HumanEval	3.91	3.30	**4.39**
MBPP	3.99	3.51	**4.79**
MT-Bench	3.74	3.01	**4.10**
Mean	4.28	3.62	**4.80**

Table 4. Qwen3.8-27B per-request mean acceptance length with the model's default sampling and a block size of 8, against its native MTP path and a community DSpark drafter.

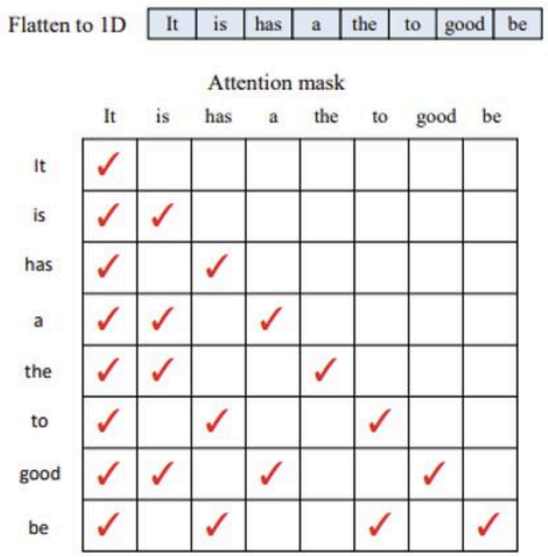
DDTree

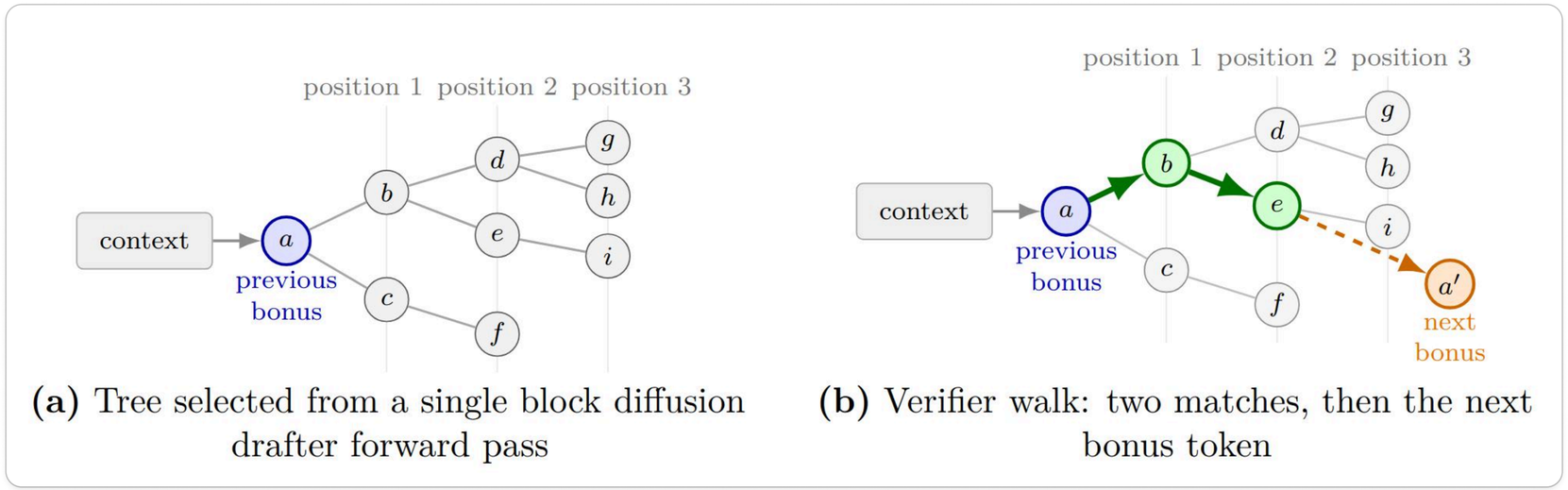
tl;dr: verify an enormous number of candidates in parallel and report big numbers



DDTree

- after our DFlash finished its forward pass, let's take the token probabilities somehow and construct a tree
- that tree can then be verified in a single forward pass using TreeAttention



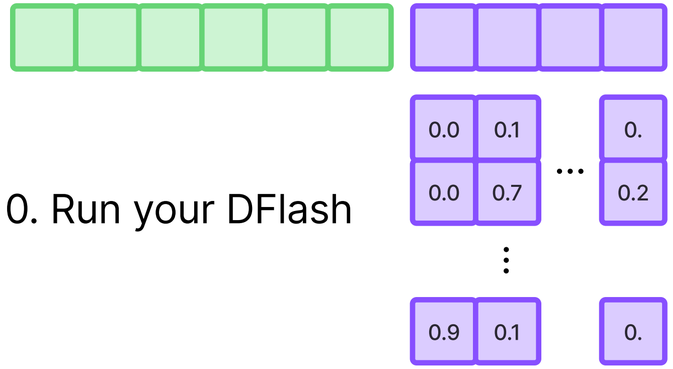


(a) Tree selected from a single block diffusion drafter forward pass

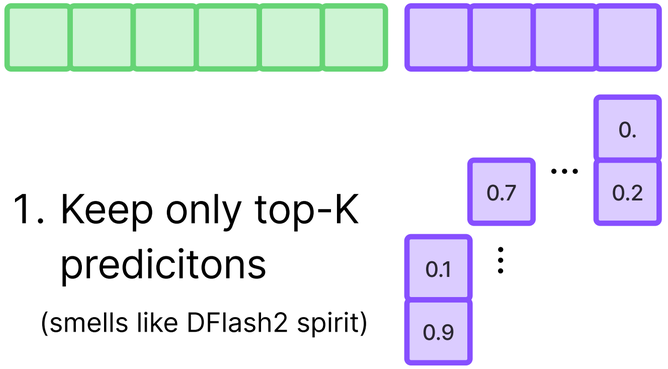
(b) Verifier walk: two matches, then the next bonus token

DDTree

How to build the tree?

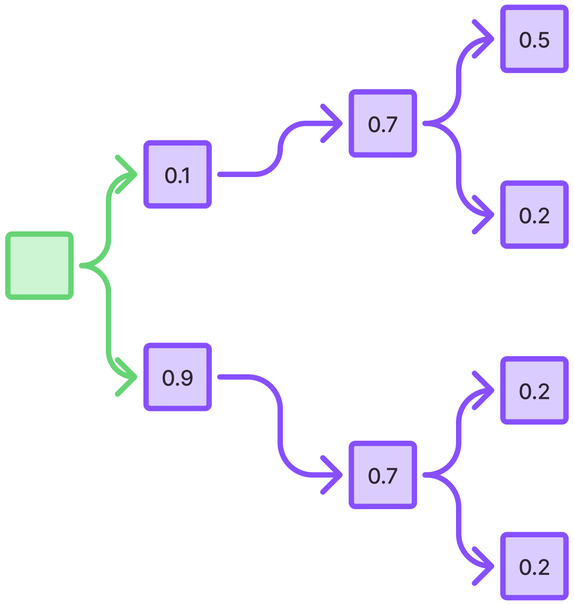


0. Run your DFlash

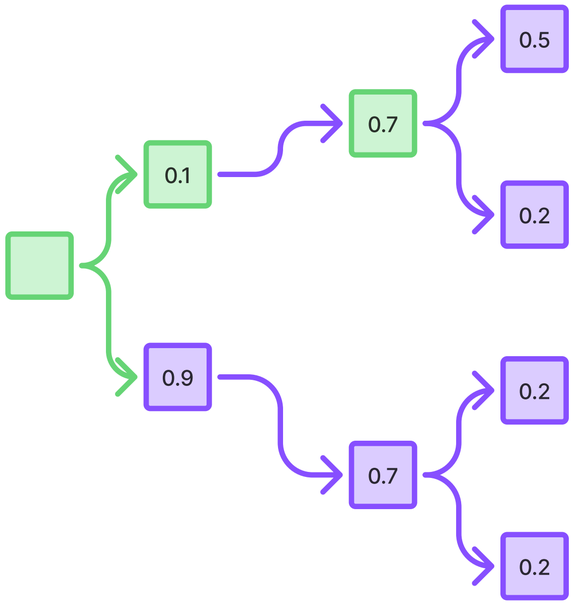


1. Keep only top-K predicitons

(smells like DFlash2 spirit)



2. Construct a tree of size B that maximizes sum of the path probabilities

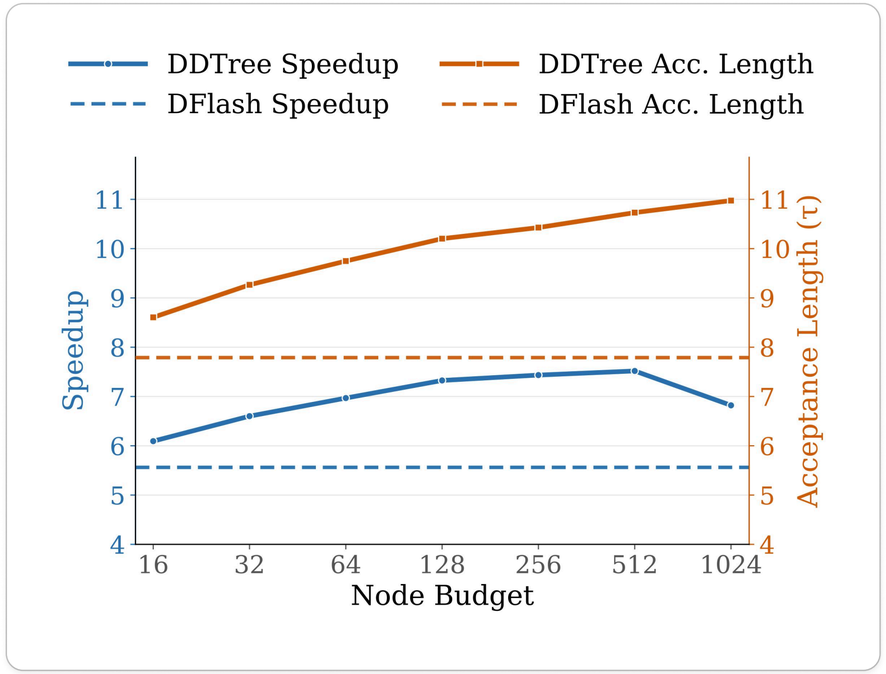


3. Verify in a single forward pass

DDTree

How to build the tree?

With big tree the increase in accepted length is expected:



But even for small block size speedup is noticeable, so e.g. you can verify not exactly 16 argmax tokens from your DFlash but smth different and win from it.

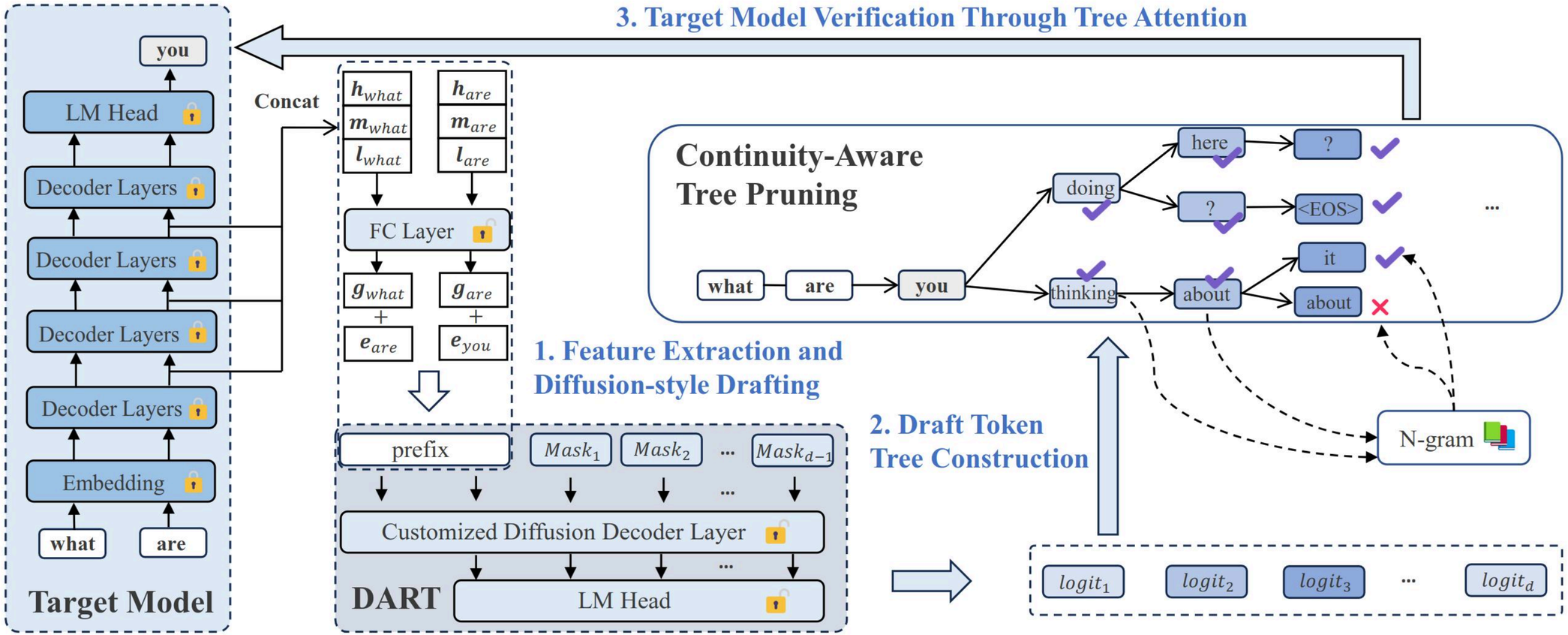
Keep in mind: all evals are with batch size = 1:

```python
for idx in tqdm(indices, disable=not dist.is_main()):
    instance = dataset[idx]
    messages = []
    for user_content in instance["turns"]:
        messages.append({"role": "user", "content": user_content})
        input_text = tokenizer.apply_chat_template(
            messages,
            tokenize=False,
            add_generation_prompt=True,
            enable_thinking=False,
        )
        input_ids = tokenizer.encode(input_text, return_tensors="pt").to(target.device)
```

(benchmark.py from their repo)

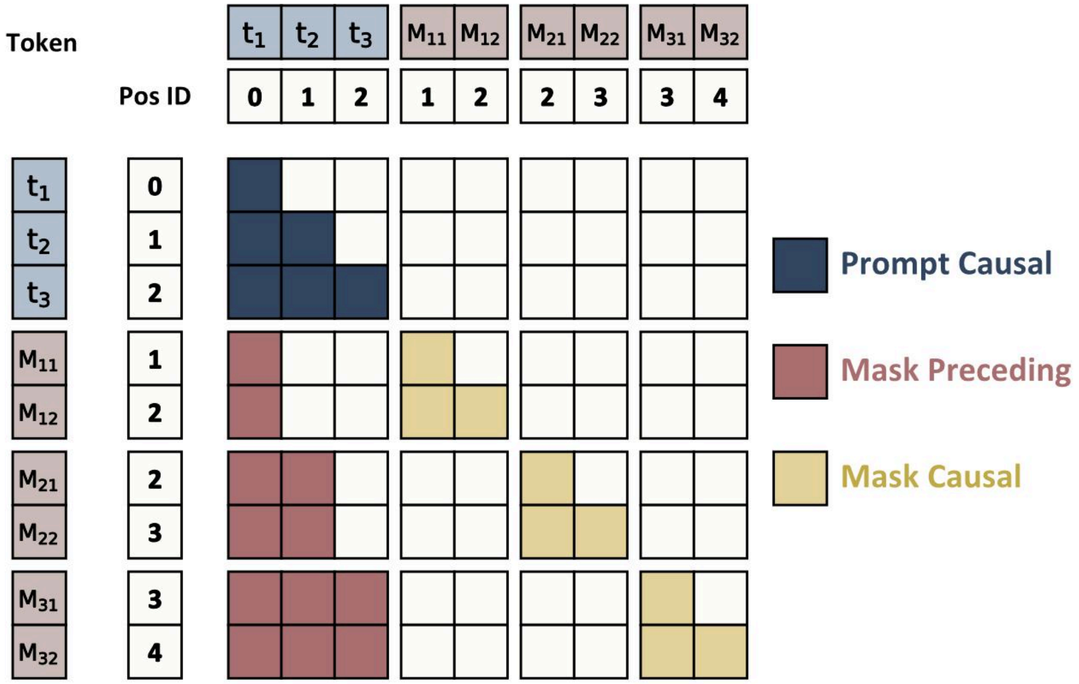
DART

- Yet another “definetely NOT just DFlash”
- Main contribution: tree pruning via N-grams



DART

- attention is causal even inside the draft block



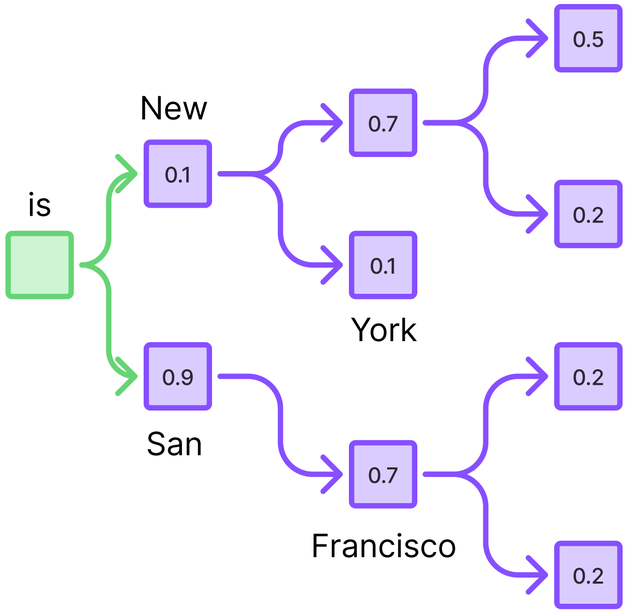
- shifted logits: distribution predicted from the features of i-th token describes the i+1-th token - like in any AR model; motivated by convincing ablation:

Table 3. Accuracy of multiple future positions under shifted and unshifted logits prediction. Hit@k denotes the proportion of cases where the ground-truth token appears among the top-k predictions ranked by model logits. α-k denotes the prediction accuracy at the k-th future position.

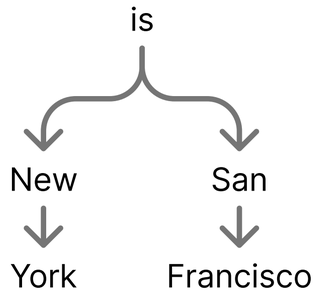
Position	α-1	α-2	α-3
Unshifted	57.7%	38.1%	25.1%
Shifted	**71.1%** (↑13.4)	**41.0%** (↑2.9)	**27.6%** (↑2.5)
Unshifted(hit@10)	87.1%	74.2%	63.2%
Shifted(hit@10)	**93.2%** (↑6.1)	**76.7%** (↑2.5)	**65.9%** (↑2.7)

DART

How to use N-gram features?



Your tree from DDTree



Your 3-gram Dolma3 Mix trie

P(New Francisco) < P(New York)

Results of reweighting

DART

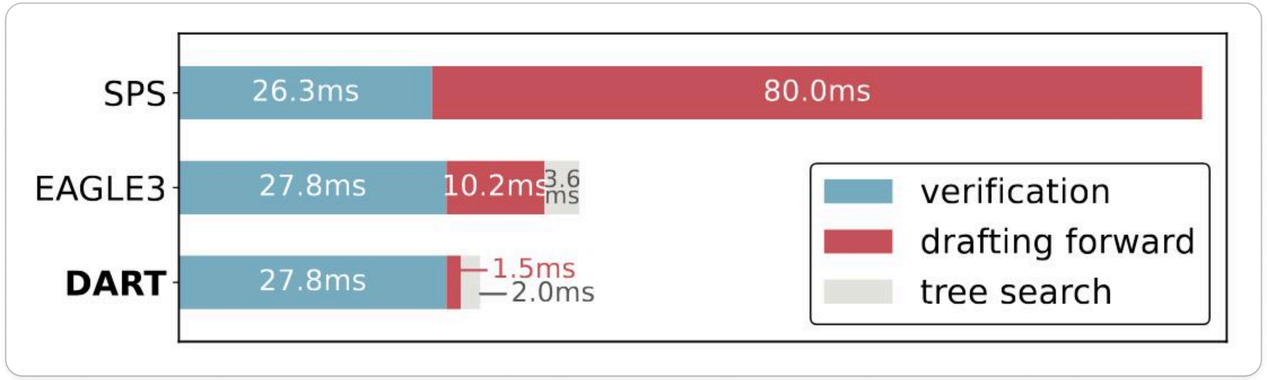
How to use N-gram features?

Table 5. Throughput improvements under larger batch sizes on the HumanEval benchmark, evaluated on H20-3e device, for EAGLE3 and DART relative to autoregressive decoding (without speculative decoding) when target model is Qwen3-4B.

Batch Size	2	4	8	16	24	32	48	64
EAGLE3	1.84×	1.69×	1.48×	1.32×	1.29×	1.28×	1.24×	1.22×
DART	**2.16**×	**2.01**×	**1.77**×	**1.57**×	**1.51**×	**1.48**×	**1.47**×	**1.45**×

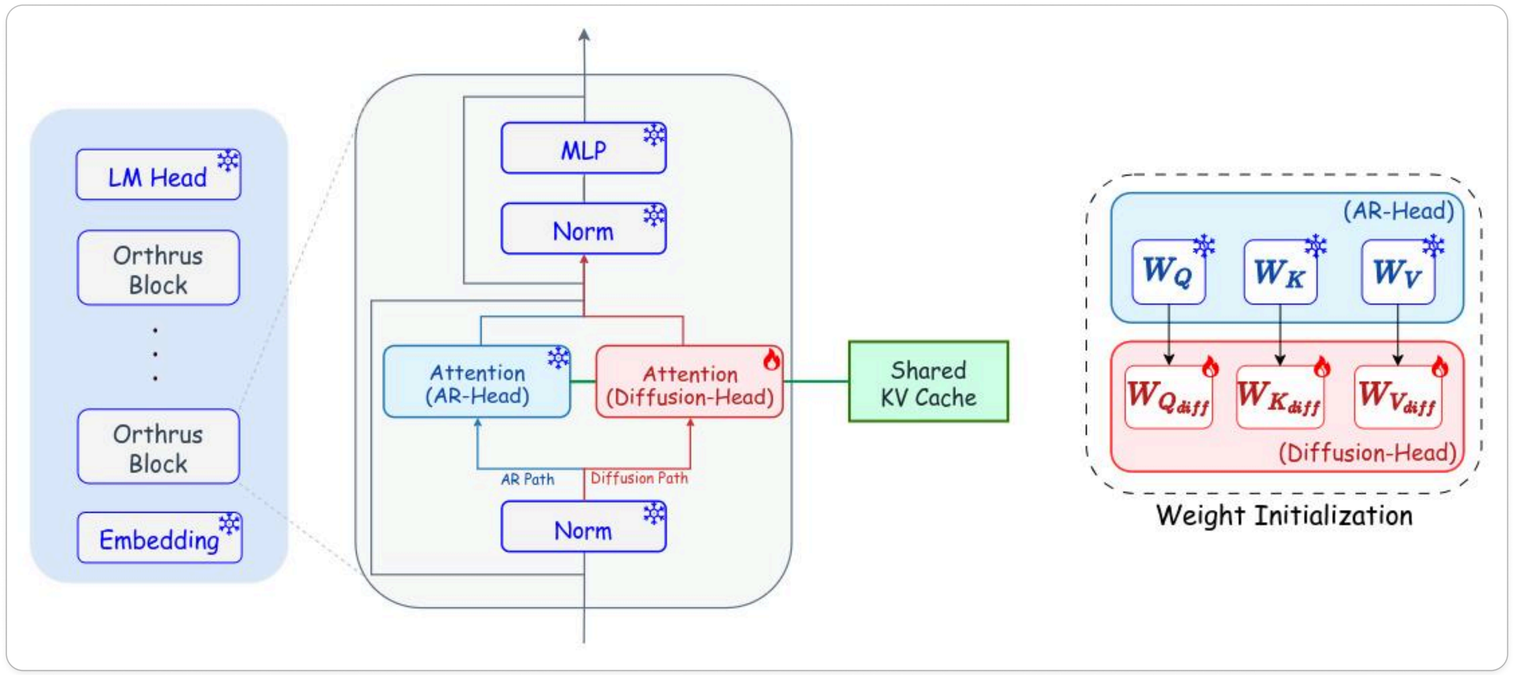
Table 2. Effect of N-gram pruning on drafting efficiency. Reported values are the average number of accepted draft tokens τ measured under speculative decoding on each benchmark.

-	HumanEval	Alpaca	Math500	CodeAlpaca
w/o N-gram	3.13	2.76	2.89	3.85
w/ N-gram	**3.63** (↑0.5)	**3.26** (↑0.5)	**3.37** (↑0.48)	**4.59** (↑0.74)



Orthrus

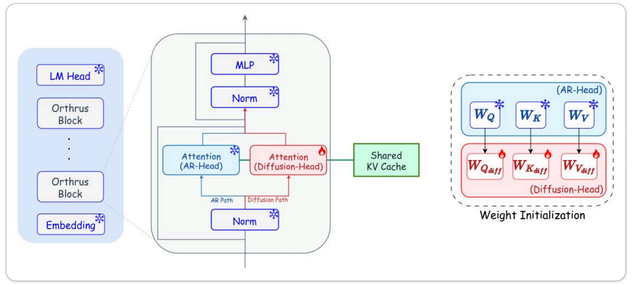
Drafter and the verifier now live inside the same backbone:

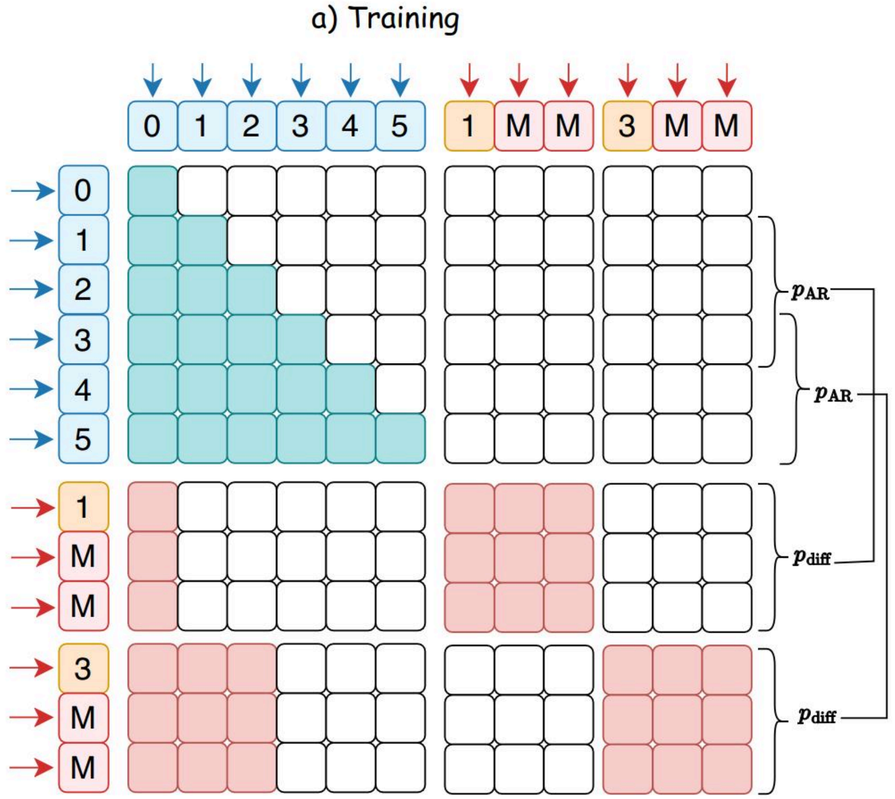


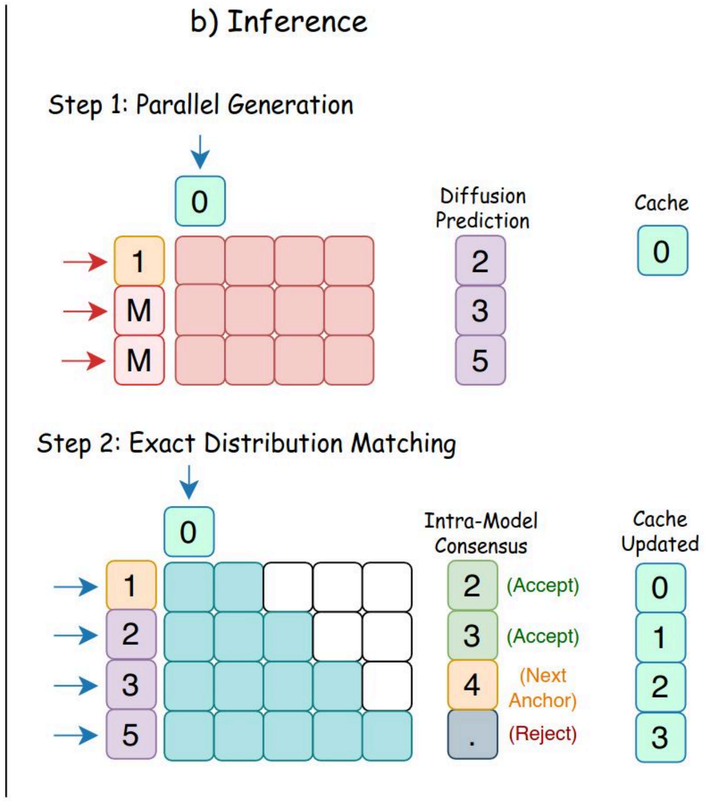
Orthrus

UNIVERSITY OF OREGON

- take pretrained AR model
- initialize new attention heads
- they are your drafter, you train them on Nemotron posttrain v2 dataset (just like DFlash)







Orthrus

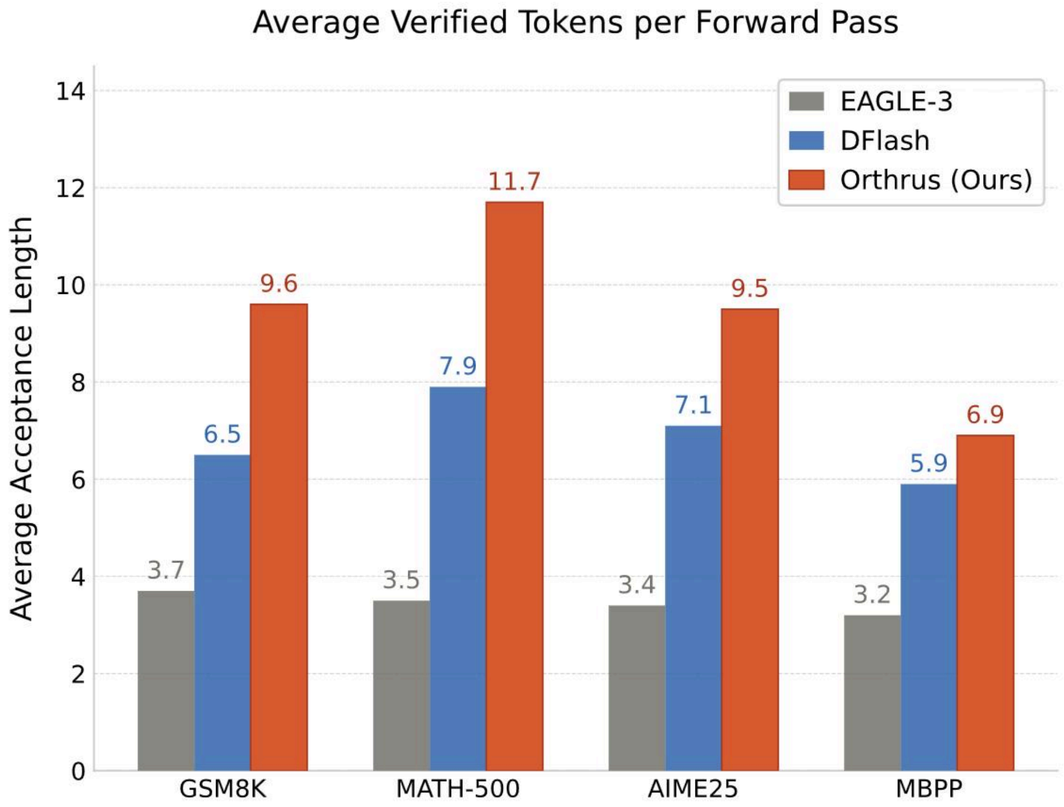
- TPF: tokens per forward pass
- worst-case TPF is 0.5

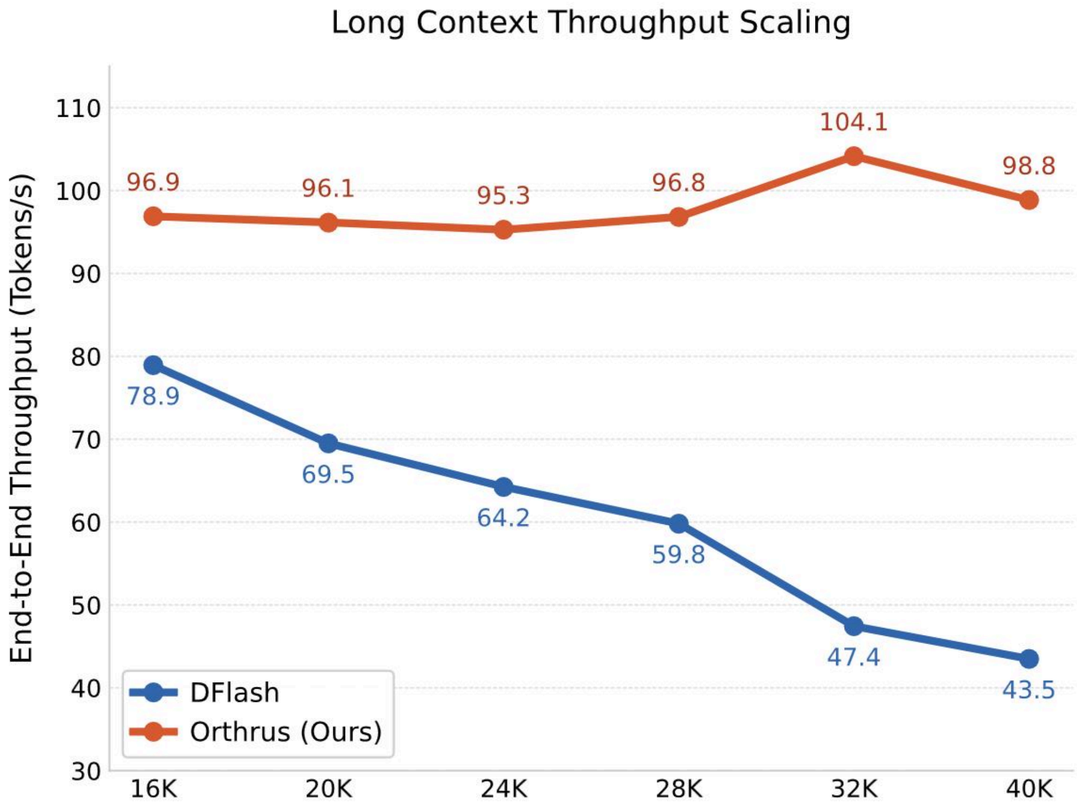
Table 1: **Efficiency Benchmarking.** We report Tokens Per Forward Pass (TPF) and relative speedup against the sequential AR baseline (which operates at **TPF** = 1.0 and 1.0× speedup) for both greedy decoding (temperature $T = 0$) and diverse sampling (temperature $T = 1$).

Task	Qwen3-1.7B				Qwen3-4B				Qwen3-8B			
	$T = 0$		$T = 1$		$T = 0$		$T = 1$		$T = 0$		$T = 1$	
	TPF	**Speedup**	**TPF**	**Speedup**	**TPF**	**Speedup**	**TPF**	**Speedup**	**TPF**	**Speedup**	**TPF**	**Speedup**
GSM8K	4.20	4.37×	3.88	4.04×	4.91	4.48×	4.97	4.53×	5.30	5.22×	4.69	4.62×
MATH-500	4.71	4.74×	4.47	4.50×	6.02	5.76×	5.62	5.38×	6.35	5.95×	5.80	5.43×
AIME24	4.33	5.65×	3.84	5.01×	5.00	6.06×	4.70	5.70×	5.63	6.81×	4.7	5.68×
AIME25	3.89	4.80×	3.55	4.38×	5.17	5.86×	4.37	4.95×	5.25	6.03×	4.40	5.05×
HumanEval	2.75	3.07×	2.60	2.90×	3.57	3.67×	3.37	3.46×	3.94	3.68×	3.37	3.15×
MBPP	2.76	3.07×	2.88	3.20×	3.74	4.43×	3.50	4.15×	3.95	3.90×	3.75	3.70×
Pseudo2code	4.60	4.90×	4.37	4.65×	7.40	7.38×	6.95	6.93×	7.51	**7.83**×	7.29	7.60×
LiveCodeBench-v5	3.86	5.87×	3.58	5.44×	4.50	6.28×	4.28	5.97×	5.17	6.68×	4.34	5.61×
Average	**3.89**	**4.25**×	**3.65**	**4.27**×	**5.04**	**5.20**×	**4.72**	**5.13**×	**5.39**	**5.36**×	**4.43**	**5.02**×

Orthrus UNIVERSITY OF OREGON

Number of accepted tokens is enormous, but who cares





Orthrus

Apparent story how AR beats block diffusion (no way):

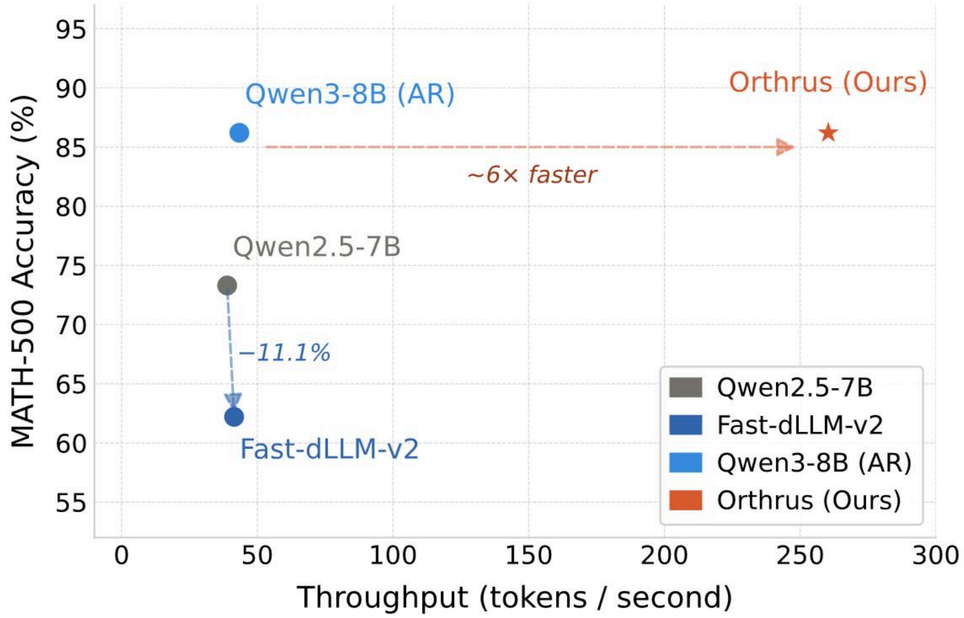
Table 2: Performance comparison with SOTA Diffusion Models.

Model	Params	GSM8K	MATH-500	AIME-24	AIME-25	HumanEval	MBPP
Dream-7B	7B	79.3	39.6	-	-	57.9	58.8
LLaDA-1.5	7B	82.4	57.4	-	-	52.4	42.8
SDAR-Qwen3-8B	8B	91.7	78.6	10.0	10.0	78.7	72.0
SDAR-Qwen3-30B-A3B	30B	91.4	77.8	16.7	10.8	71.6	87.2
Fast-dLLM-v2	7B	83.7	61.5	-	-	43.3	28.2
Mercury Coder Small	-	-	-	-	-	90.0	76.6
Gemini Diffusion	-	-	-	-	20.0	89.6	76.0
Orthrus-Qwen3-8B	**8.7B**	**96.0**	**86.2**	**28.3**	**23.3**	**95.1**	**93.4**

Denoising in >1 steps does not bring any benefit:

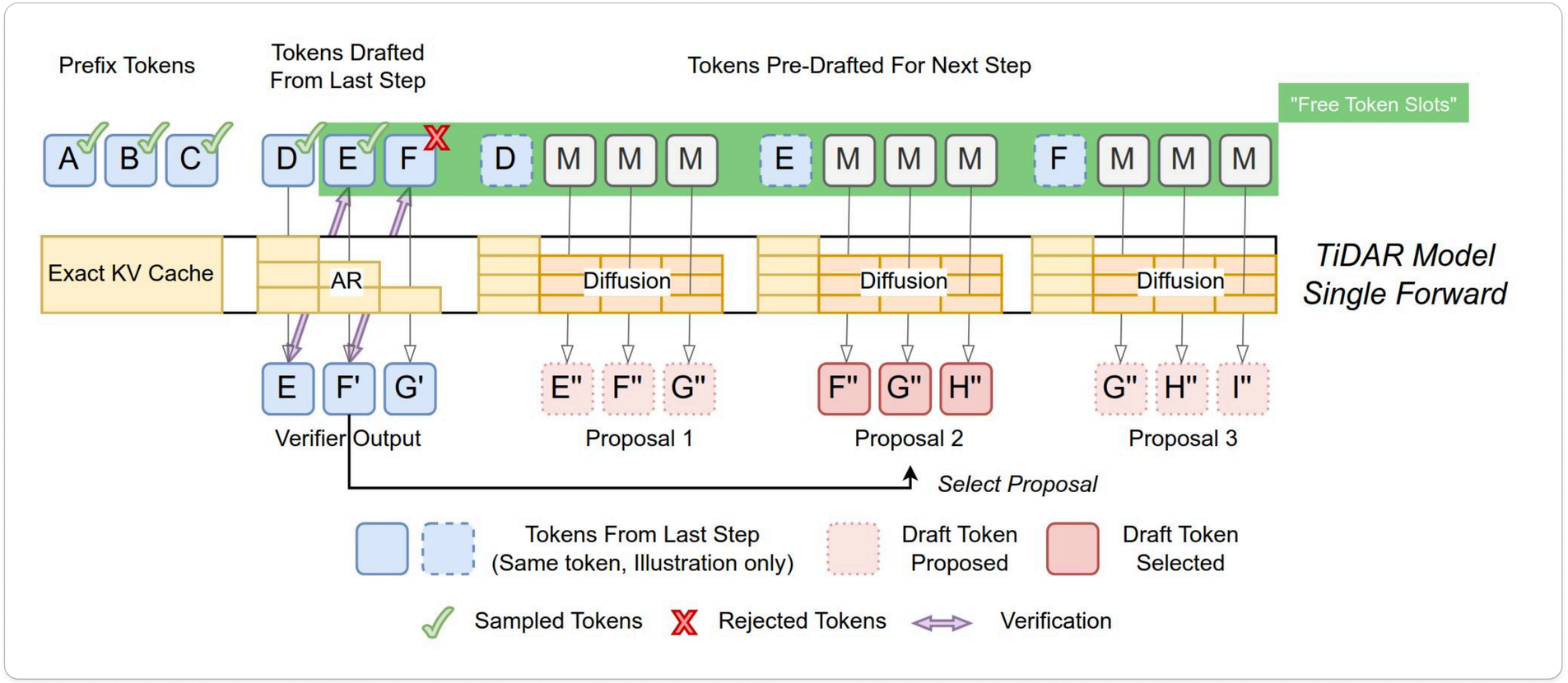
Table 3: Impact of multi-step denoising.

Strategy	Passes	TPF
Multi-Step	2	3.53
Orthrus (Single-Step)	**1**	**6.35**



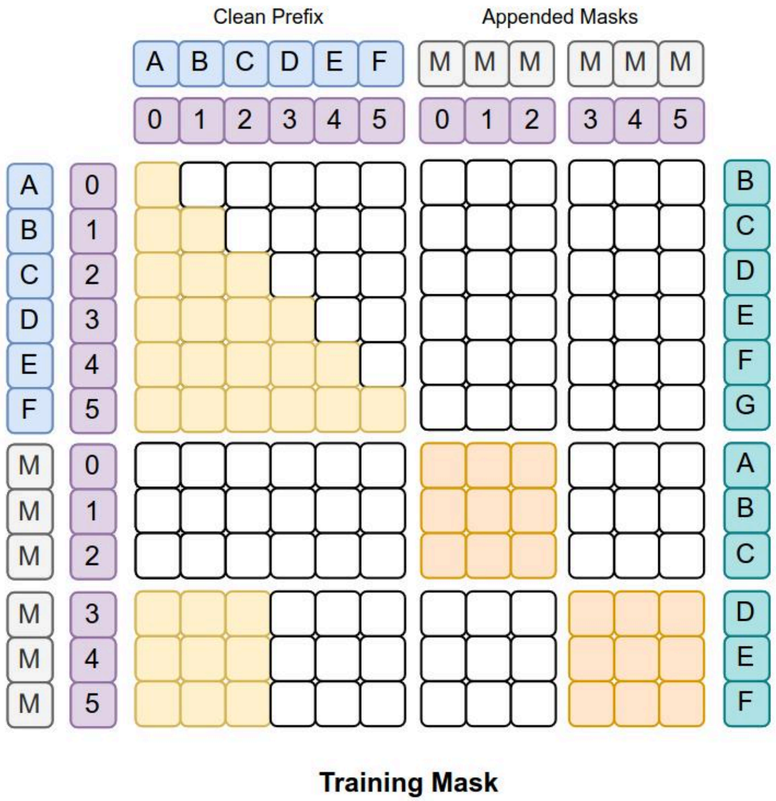
TiDAR NVIDIA

Yet another “definitely NOT just Orthrus”



TiDAR NVIDIA

- train AR and drafter simultaneously



Training Mask

TiDAR NVIDIA

Inference:

- draft k tokens
- construct all possible continuations with mask tokens appended to different stopping locations
- verify and draft the new tokens simultaneously: AR at verification, full attention at draft part
- pick the correct draft candidate